

Software Design Document

Synthetic Speech & Voice-Spoofing Detection System

Decision Support for Fraud Prevention through AI-Based Spoofed-Audio Detection

Lab 5 — Designing Our Own Project's Architecture

Project	Synthetic Speech & Voice-Spoofing Detection System (AI course semester project, MUST University)
Document type	Software Design Document (IEEE 1016-style structure) with the Lab 5 practical-session deliverables
Design baseline	SRS — Voice-Spoofing Detection System, v3.0 (Merged), Lab 3 requirements baseline
UML standard	UML 2.5.1
Target platform	Python 3.11 · PyTorch / torchaudio · scikit-learn · FastAPI · Streamlit · SQLite · Docker · Colab / Kaggle · Weights & Biases · GitHub Actions
Group members	Donia Mabrouk · Iheb Zemzemi · Malak Ben Salem
Version	1.0 — September 2026

This document converts the Lab 3 requirements baseline into a coherent, traceable and implementation-ready software design for our own project, and completes the Lab 5 practical-session deliverables: methodology and architecture decision, component model, class model, sequence models, design-pattern justification, review log and the SDD skeleton.

Table of Contents

1. Introduction

1.1 Purpose

1.2 Scope

1.3 Definitions, Acronyms and Abbreviations

1.4 References

2. System Architecture

2.1 Chosen Methodology and Architectural Style

2.2 Component Diagrams

2.3 Component Responsibility Descriptions

2.4 Deployment View

3. Detailed Design

3.1 Class Diagrams

3.2 Class Descriptions

3.3 Object-Oriented Quality

4. Data Design

4.1 Data Model

4.2 Key Data Structures

4.3 Data Security and Lifecycle

5. Interface Design

5.1 User Interface Overview

5.2 API and Interface Contracts

6. Behavioral Design

6.1 Sequence Diagrams for Key Use Cases

6.2 Additional Behavioral Rules

7. Design Rationale

7.1 Design Patterns Used and Why

7.2 Alternatives Considered and Trade-offs

8. Requirements Traceability Matrix

9. Appendices

9.1 Design Quality Checklist

9.2 Cross-Diagram Consistency

9.3 Internal Pre-Review Log

9.4 Peer Feedback Received (in-class exchange)

9.5 Assumptions and Open Decisions

9.6 Sequence Templates (Lab 5 worksheet format) 29

9.7 Lab 5 Submission Checklist and Instructor Checkpoints 30

List of Figures

Figure 1 — UML component diagram of the online inference and analysis tier 7

Figure 2 — UML component diagram of the research tier (training and evaluation) 8

Figure 3 — Specification-level UML class diagram — analysis domain 11

Figure 4 — UML class diagram — evaluation and research-governance domain 12

Figure 5 — UML sequence diagram for UC-01 Analyze Audio via Dashboard 20

Figure 6 — UML sequence diagram for UC-08 Run External Generalization Test 21

Document Control

Table 1 — Document revision history

Version	Date	Description	Authors
1.0	September 2026	Lab 5 design: architecture, class and sequence models, pattern justification, traceability, review appendices and SDD skeleton.	Donia Mabrouk, Iheb Zemzemi, Malak Ben Salem

Executive Summary

The design applies **object-oriented design** inside a **layered architecture built around one shared core library**. The core (Audio Intake, Preprocessing, Detector Runtime, Risk & Confidence) is used unchanged by two runtime tiers: an **online tier** (Docker: REST API and analyst dashboard) and a **research tier** (Colab / Kaggle notebooks: dataset audit, training, evaluation and the external generalization test). Because both tiers share the same preprocessing profile and detector contract, results measured offline describe the system that is actually deployed.

Three decisions distinguish it from a generic ML service. **(1) Research rules are structure, not convention:** a ModelBundle binds model, preprocessing profile and risk policy and becomes immutable when frozen; an External Test Guard backed by an append-only run registry allows the DEEP-VOICE test to run only for a frozen bundle and only once. **(2) One path for every channel:** the dashboard calls the same API Gateway as external API Clients, so both always apply the same input policy and risk logic. **(3) Decision support is built into the result object:** an *uncertain* outcome is a normal result, and every result carries a decision-support notice.

The design comprises 18 components, 17 classes in two domain models, two principal sequence models (UC-01 and UC-08), six patterns and a traceability matrix (Table 14) covering all 40 SRS requirements (20 functional, 11 non-functional, 9 project/research constraints). Section 9.7 maps every Lab 5 deliverable and checkpoint to where it appears here; the peer-review exchange and instructor sign-offs are completed in class and are marked as such.

1. Introduction

1.1 Purpose

This Software Design Document (SDD) defines how the Synthetic Speech & Voice-Spoofing Detection System will realize the behavior, quality targets and research constraints specified in the project SRS (v3.0, Lab 3). It establishes the architecture, components, domain classes, data design, interfaces, interaction scenarios, design patterns and traceability that guide implementation (Weeks 2–12) and testing (Week 13). It is also our Lab 5 submission: it applies the Lab 4 design steps to our own project instead of a case study, and Sections 1–2 are the SDD skeleton required at the end of the practical session.

1.2 Scope

The design covers analysis of short recorded speech submitted through the analyst dashboard or a REST API; input validation; a versioned preprocessing profile; a CNN detector with an MFCC baseline used for evaluation; a three-way prediction (authentic, synthetic, uncertain) with confidence and a LOW/MEDIUM/HIGH risk level; analysis metadata logging, history and annotation; an evaluation harness; the frozen-model external generalization test on DEEP-VOICE; a model-bundle registry; operational service logs; and containerized deployment.

Out of scope (SRS 1.2): live real-time telephony or streaming detection; defense against adversarial evasion of the detector (documented as a limitation); fine-tuning of pretrained front-ends and spectrogram explainability; a second external test dataset; production multi-tenant identity management, downstream case management and the choice of a commercial hosting vendor.

Target platform. Unlike the Lab 4 case study, our technology stack is fixed by the project plan (cover page). The component contracts nevertheless stay technology-independent, so the parts the SRS deliberately leaves open (access-control mechanism, persistence engine, hosting) remain replaceable. No performance figures are asserted: latency, size, duration and retention limits remain the named parameters of SRS 7.4 (Section 9.5).

1.3 Definitions, Acronyms and Abbreviations

Table 2 — Key terms

Term	Meaning
SRS / SDD	Software Requirements Specification (Lab 3, v3.0) / Software Design Document (this document)
FR / NFR / PC	Functional Requirement / Non-Functional (runtime quality) Requirement / Project or Research Constraint, as defined in the SRS
OOD, UML	Object-Oriented Design; Unified Modeling Language 2.5.1
Online tier	Containerized runtime: analyst dashboard, REST API and the services behind it
Research tier	Offline runtime: Colab / Kaggle notebooks for dataset audit, training, evaluation and the external test
Shared core library	Preprocessing, detector, risk-scoring and model-registry code used unchanged by both tiers
Model bundle	One versioned unit binding a detector, its preprocessing profile and its risk policy (NFR-09)
Frozen model	A bundle whose weights, profile and decision thresholds may no longer change (PC-04)
Preprocessing profile	Documented, versioned audio processing: 16 kHz resampling, silence trimming, feature extraction
Risk policy	Versioned thresholds and mapping from score to prediction, confidence and LOW/MEDIUM/HIGH risk level
Decision support	The system assists a human judgment; it never issues an automatic or definitive fraud verdict (NFR-05)
Group-aware split	A data split in which all clips from one speaker or source recording fall in the same partition (PC-02)
FoR-2sec / DEEP-VOICE	Development dataset (TTS speech, 2-second clips) / frozen external test dataset (RVC voice conversion)
TTS / RVC	Text-to-speech / Retrieval-based Voice Conversion
EER / ROC-AUC	Equal Error Rate / Area Under the ROC curve

1.4 References

- [1] SRS — Voice-Spoofing Detection System, v3.0 (Merged team edition), Lab 3, September 2026 (source of every FR-, NFR- and PC- identifier used here).
- [2] Project Proposal & Requirements Document; Execution Reference — 14-week SDLC roadmap (internal team documents).
- [3] Lab 4 Design and Methodology, and Lab 5 Practical Session handouts, Dr. Soura Boulaares, MUST University.
- [4] Software Design Document, Campus Library Management System — Lab 4 (our group's case-study SDD, used as the structural template).
- [5] UML Diagrams Professional Reference Guide, UML 2.5.1; IEEE Std 1016-2009, Software Design Descriptions.
- [6] E. Gamma, R. Helm, R. Johnson, J. Vlissides, *Design Patterns*, Addison-Wesley, 1994; M. Fowler, *Patterns of Enterprise Application Architecture*, 2002 (Repository).
- [7] Dataset documentation: Fake-or-Real (FoR) 2-second variant; DEEP-VOICE and its accompanying study (as cited in the SRS).

2. System Architecture

2.1 Chosen Methodology and Architectural Style

What kind of system is this? It is ML- and data-heavy in its offline research lifecycle and request/response in its online lifecycle: it analyzes short recordings synchronously (not as a live stream) and it is not made of services that must scale independently. Three properties drive the design:

- **Two lifecycles, one core.** Training, evaluation and live inference must apply the same preprocessing and the same detector contract (FR-04, NFR-09); otherwise numbers measured offline would not describe the deployed system.
- **Rules that must hold structurally.** Tuning on FoR-2sec only (PC-03), freezing before the external test (PC-04), evaluating DEEP-VOICE exactly once (PC-05) and deploying only the CNN (PC-08) are too important to rely on team discipline.
- **Small team, fixed timeline.** Three members, 14 weeks and free-tier compute: the design must be testable in isolation (NFR-11) and operable without infrastructure a student team cannot run.

Table 3 — Methodology and architecture comparison

Option considered	Pros for our project	Cons for our project	Chosen?
Structured design (functional decomposition / script pipeline)	Natural for ML scripts (preprocess → train → evaluate); fastest to start.	Versioned bundles, policies and guards are stateful entities; process rules end up scattered across scripts; poor fit for the API and dashboard.	No
Object-oriented design	Models ModelBundle, RiskPolicy, InputPolicy, EvaluationRun as objects with encapsulated state; polymorphic Detector; freeze and one-time rules live in code.	Tensor code is naturally functional; risk of over-engineering if every helper becomes a class.	Selected (methodology)
Layered (n-tier) architecture	Separates presentation, API, application services, core and persistence; dependencies point inward; maps to the repository folders; isolated tests.	Some indirection for a small system.	Selected (style)
Pipe-and-filter	Matches audio → features → score → decision; each stage testable.	Does not describe the dashboard, API, logging or the evaluation governance.	Partial: inside Analysis Service
Component-based	Named provided interfaces make parts replaceable (Detector, storage, access control).	A full plug-in model is unnecessary at this size.	Partial: interfaces at boundaries
Client-server	True at the network level (browser or API Client and the API container).	Too coarse to describe the internal structure.	Partial: deployment context
Microservices	Detector could scale independently.	Operational overhead for three students; no scaling requirement in the SRS; harder debugging.	Rejected
Event-driven	Useful for streaming detection.	Analysis is synchronous on 2-second-scale clips; streaming is out of scope (SRS 1.2).	Rejected for this release

Methodology & Architecture Decision

Methodology: object-oriented design. **Architectural style:** layered (n-tier) architecture around a shared core library, deployed as two runtime tiers (online and research), with pipeline stages inside the Analysis Service and component-style interfaces at every layer boundary.

Justification. Our system's value depends on rules that must be enforced structurally — identical preprocessing at training and inference (FR-04, NFR-09) and a frozen model evaluated exactly once on DEEP-VOICE (PC-04, PC-05) — so we model bundles, policies and guards as objects with encapsulated state rather than as loose scripts. A layered architecture with one shared core keeps the offline and online lifecycles from drifting apart while staying simple enough for three people, 14 weeks and free-tier compute. Microservices and event-driven styles were rejected because nothing in the SRS requires independent scaling or asynchronous processing.

Architectural principles

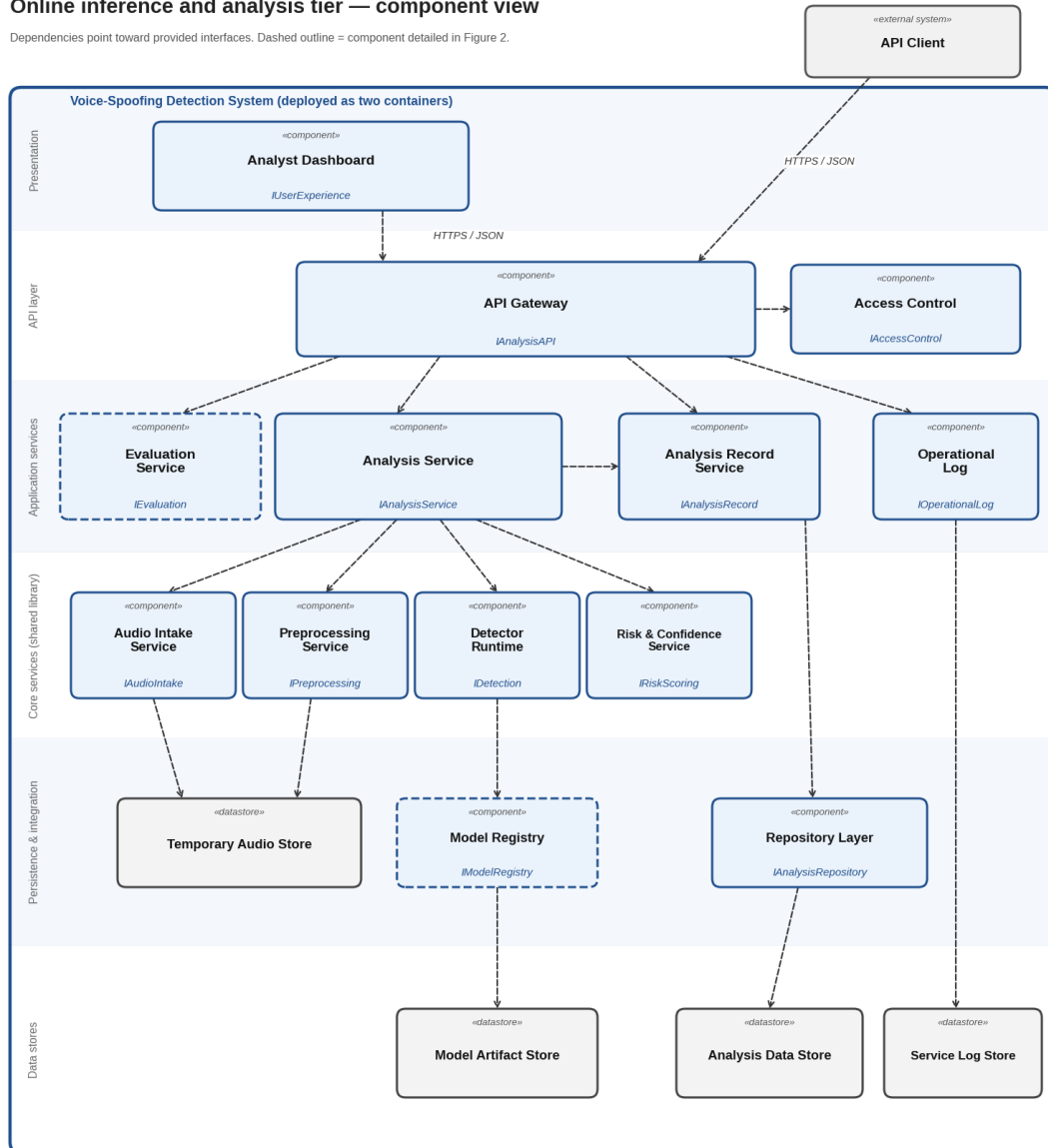
- Dependencies point inward toward stable interfaces; core logic never depends on the browser, the database or a vendor.
- Each component has one principal responsibility and one named provided interface (Table 4); external systems are reached only through adapters.
- Research-integrity rules are enforced on the code path (frozen flag, guard, run registry), not documented and hoped for.
- Security, privacy and observability are cross-cutting: no raw audio in logs, no internals in errors, every result traceable by request ID and version.

2.2 Component Diagrams

Figure 1 shows the online tier as architectural layers; Figure 2 shows the research tier. The Analyst Dashboard and external API Clients meet at the API Gateway, which is the only way into the services. Components marked as shared (Preprocessing Service, Detector Runtime, Risk & Confidence Service, Model Registry) appear in both figures because the same code runs in both tiers.

Online inference and analysis tier — component view

Dependencies point toward provided interfaces. Dashed outline = component detailed in Figure 2.



Legend: dashed open arrow = dependency on a provided interface; dashed outline = component detailed in Figure 2; shaded bands = architectural layers.

The Analyst Dashboard reaches the detector only through the API Gateway, so dashboard and API Clients cannot disagree (NFR-08).

Figure 1 — UML component diagram of the online inference and analysis tier

The online tier contains no cyclic dependencies. The Analysis Service is a facade over the core stages, so the fixed order (intake, preprocessing, detection, risk decision, record) is defined once and used by every channel. The Repository Layer and the Model Registry isolate storage, and the External Test Guard and Experiment Tracker do not appear online at all: the deployed system can only load a bundle that the research tier has already frozen.

Research tier (offline) — component view

Arrows into a group are representative dependencies on the group's provided interfaces.

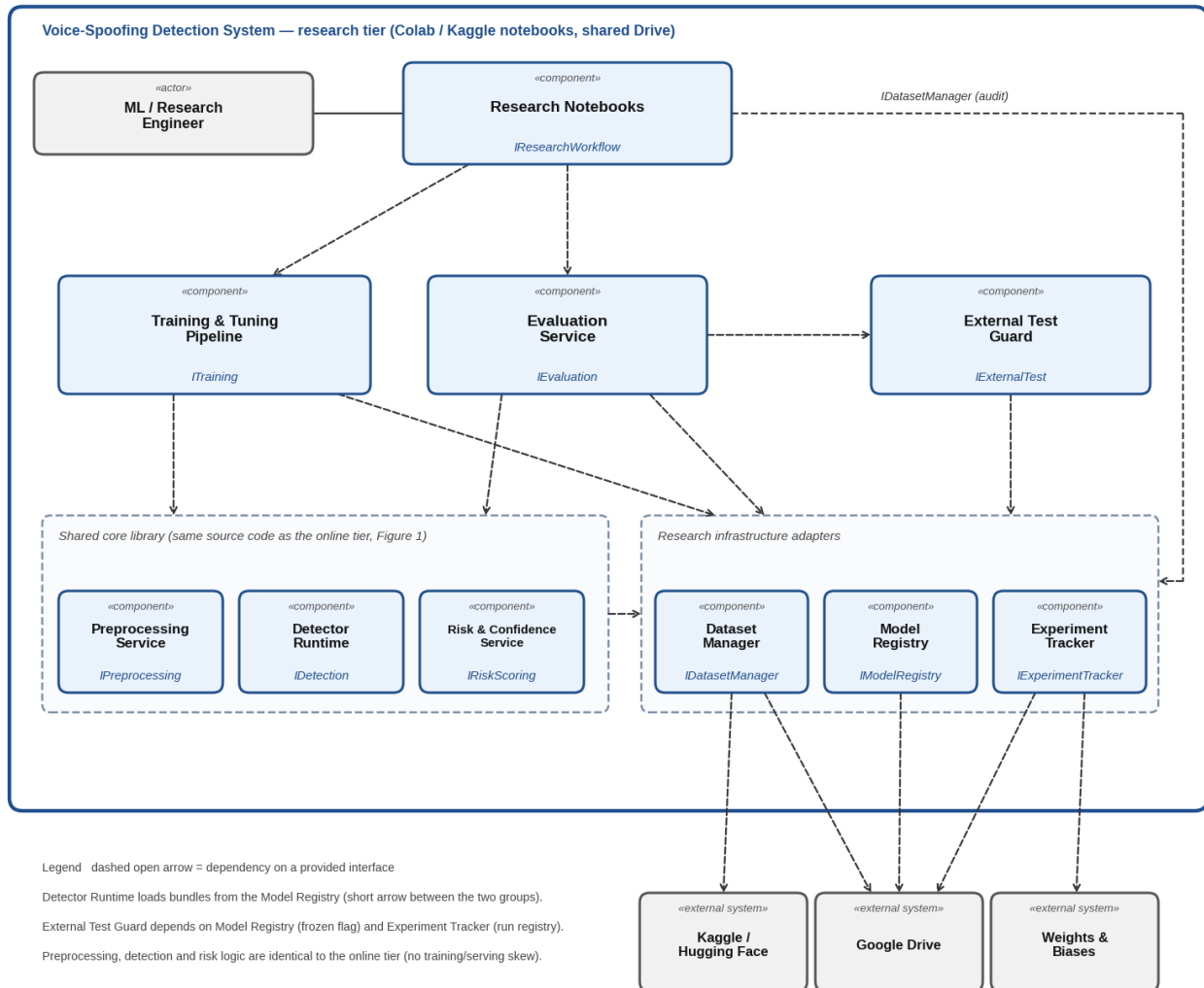


Figure 2 — UML component diagram of the research tier (training and evaluation)

The research tier reuses the shared core through its provided interfaces. The Evaluation Service asks the External Test Guard for authorization before touching DEEP-VOICE, and the Guard consults the Model Registry (is the bundle frozen?) and the Experiment Tracker (has an external run already been registered?). Training and tuning use only FoR-2sec partitions supplied by the Dataset Manager. The tier has no user interface of its own: the ML / Research Engineer drives it from notebooks.

2.3 Component Responsibility Descriptions

Table 4 — Component responsibility model (18 components)

Component	Responsibility	Provided interface	Talks to
Analyst Dashboard Online · Presentation	Lets the Fraud Prevention Analyst submit a recording, read prediction, confidence, risk level and decision-support notice together, browse flagged history and annotate results. Holds no detection logic and never calls the detector directly (FR-01, FR-08, FR-10, FR-11, NFR-05, NFR-08).	<i>IUserExperience</i>	API Gateway
API Gateway Online · API	Single HTTP entry point for the dashboard and for API Clients: assigns a request ID, routes requests, translates failures into structured errors without stack traces and emits request-level operational events (FR-02, NFR-02, NFR-06, NFR-07, NFR-08).	<i>IAnalysisAPI</i>	Access Control; Analysis Service; Analysis Record Service; Evaluation Service; Operational Log
Access Control Online · API	Authenticates the requester and authorizes protected actions: annotation (analyst accounts only), service-log review and evaluation reports. The mechanism (API key or analyst login) is an open decision (SRS 7.4) hidden behind this interface (FR-11, NFR-02).	<i>IAccessControl</i>	— (called only)
Analysis Service Online · Application	Orchestrates the shared 'Validate & Analyze Audio' use case in a fixed order (intake, preprocessing, detection, risk decision, record), measures latency and aborts without recording anything when a step fails (UC-03, FR-05, NFR-01, NFR-06).	<i>IAnalysisService</i>	Audio Intake Service; Preprocessing Service; Detector Runtime; Risk & Confidence Service; Analysis Record Service
Audio Intake Service Online · Core	Accepts an uploaded file, validates readability, format, size and duration against the published input policy, holds the raw audio only temporarily and purges it after RETENTION_HOURS (FR-03, NFR-03, NFR-08).	<i>IAudioIntake</i>	Temporary Audio Store
Preprocessing Service Shared · Core	Applies the versioned preprocessing profile (16 kHz resampling, silence trimming, feature extraction) identically for training, evaluation and live requests, which prevents training/serving skew (FR-04, NFR-09).	<i>IPreprocessing</i>	Temporary Audio Store
Detector Runtime Shared · Core	Loads a versioned model bundle from the Model Registry and returns a raw synthetic-speech score for a feature tensor; deployment accepts only the CNN, the MFCC baseline exists for evaluation (FR-05, FR-12, NFR-01, NFR-09, PC-08).	<i>IDetection</i>	Model Registry
Risk & Confidence Service Shared · Core	Converts a raw score into exactly one prediction (authentic, synthetic or uncertain), a confidence value in [0,1] and a LOW/MEDIUM/HIGH risk level using the versioned risk policy; thresholds are tuned on FoR-2sec only (FR-05, FR-06, FR-07, PC-03).	<i>IRiskScoring</i>	— (called only)
Analysis Record Service Online · Application	Stores the seven-field metadata of every successful analysis without raw audio, serves history with flagged markers and stores analyst annotations beside the unchanged original result (FR-09, FR-10, FR-11, NFR-04, NFR-07).	<i>IAnalysisRecord</i>	Repository Layer
Operational Log Online · Application	Keeps service-level events (model/config version, request volume, latency, error rate) separate from analysis metadata, with no raw audio or personal identifiers, for the operations administrator (FR-20, NFR-04).	<i>IOperationalLog</i>	Service Log Store
Repository Layer Online · Persistence	Provides transactional persistence for analysis records and annotations behind an interface, so the store (SQLite initially) can change without touching services (FR-09, NFR-11).	<i>IAnalysisRepository</i>	Analysis Data Store
Research Notebooks Research · Entry point	Colab/Kaggle notebooks used by the ML / Research Engineer to launch the dataset audit, training, evaluation and the external test. They call the same core library as the online tier and hold no experiment logic of their own (UC-07, UC-08).	<i>IResearchWorkflow</i>	Training & Tuning Pipeline; Evaluation Service; Dataset Manager
Training & Tuning Pipeline Research · Research service	Trains the MFCC baseline and the CNN on the FoR-2sec training split, tunes hyper-parameters and risk thresholds on the FoR-2sec validation split only, assembles a ModelBundle and hands it to the registry (PC-03, PC-07, PC-08).	<i>ITraining</i>	Dataset Manager; Preprocessing Service; Detector Runtime; Risk & Confidence Service; Model Registry; Experiment Tracker
Evaluation Service Research · Research service	Runs baseline and CNN through one shared harness on the same FoR-2sec test split, computes the required metric set and builds the evaluation report with a labeled external-generalization section (FR-12 to FR-19, UC-07, UC-09).	<i>IEvaluation</i>	Dataset Manager; Preprocessing Service; Detector Runtime; Risk & Confidence Service; Model Registry; External Test Guard; Experiment Tracker
External Test Guard Research · Research service	Protection point in front of the DEEP-VOICE evaluation: authorizes a run only for a frozen bundle and only if no external run is registered yet, so the test can never become a tuning set (FR-15, PC-04, PC-05).	<i>IExternalTest</i>	Model Registry; Experiment Tracker

Component	Responsibility	Provided interface	Talks to
Dataset Manager Research · Infrastructure	Owens dataset manifests: audits FoR-2sec and DEEP-VOICE, builds the group-aware split and exposes labeled splits with class counts (PC-01, PC-02, FR-14).	<i>IDatasetManager</i>	Kaggle / Hugging Face; Google Drive
Model Registry Shared · Infrastructure	Stores versioned ModelBundles (model + preprocessing profile + risk policy), marks a bundle frozen and refuses further changes, and lets the online tier load a matching set of versions (NFR-09, PC-04, PC-07).	<i>IModelRegistry</i>	Model Artifact Store; Google Drive
Experiment Tracker Research · Infrastructure	Adapter to Weights & Biases plus an append-only local run registry: logs config, seed, dataset reference and metrics for every run and remembers whether the external test has already been run (PC-05, PC-07).	<i>IExperimentTracker</i>	Weights & Biases; Google Drive

Data stores (Analysis Data Store, Temporary Audio Store, Service Log Store, Model Artifact Store) and external systems (API Client, Kaggle / Hugging Face, Google Drive, Weights & Biases) are shown in Figures 1–2 and are reached only through the components above.

2.4 Deployment View

The logical design deploys as two containers started together by docker-compose from the README instructions (NFR-10): an **api** container hosting every online service and the frozen bundle, and a **dashboard** container. The research tier runs in Colab or Kaggle notebooks that clone the same repository, so the shared core is literally the same source code. Promotion from research to online is a copy of one frozen bundle into the read-only Model Artifact Store; the Detector Runtime activates it at start-up and refuses a bundle whose model, profile and risk-policy versions do not match (NFR-09). The hosting vendor remains open.

Table 5 — Deployment responsibilities and communication paths

Node or environment	Deployed artifacts	Communication
Device of analyst or API Client	Browser (dashboard) or any HTTP client	HTTPS to the dashboard or api container
Container: dashboard	Analyst Dashboard (Streamlit) and its UI assets	HTTPS / JSON to the api container only
Container: api	API Gateway, Access Control, Analysis Service, Audio Intake, Preprocessing, Detector Runtime, Risk & Confidence, Analysis Record Service, Operational Log, Repository Layer; the active frozen bundle	Receives HTTPS; reads the Model Artifact Store (read-only); writes the three data stores below
Persistent volumes	Analysis Data Store (SQLite initially), Temporary Audio Store (with retention job), Service Log Store, Model Artifact Store	Local mounts to the api container
Research environment (Colab / Kaggle)	Research Notebooks and the same core library; Training & Tuning, Evaluation, External Test Guard, Dataset Manager, Model Registry, Experiment Tracker	HTTPS to Kaggle / Hugging Face (datasets), Google Drive (datasets, checkpoints, run registry) and Weights & Biases (tracking)
Continuous integration	GitHub Actions running the pytest suites for preprocessing, the risk and confidence layer and the API	Runs on every push and pull request to main (NFR-11)

3. Detailed Design

3.1 Class Diagrams

The domain is modeled at specification level in two views: the **analysis domain** (what is submitted, scored, decided and recorded) and the **evaluation and research-governance domain** (how runs, splits, metrics and the external-test rule are represented). Classes were derived from the nouns of the SRS requirements and use cases and operations from their verbs (validate, decide, freeze, authorize). Attributes are private and typed; operations are public; relationships use multiplicities, composition, aggregation and realization.

Analysis domain — specification-level class diagram

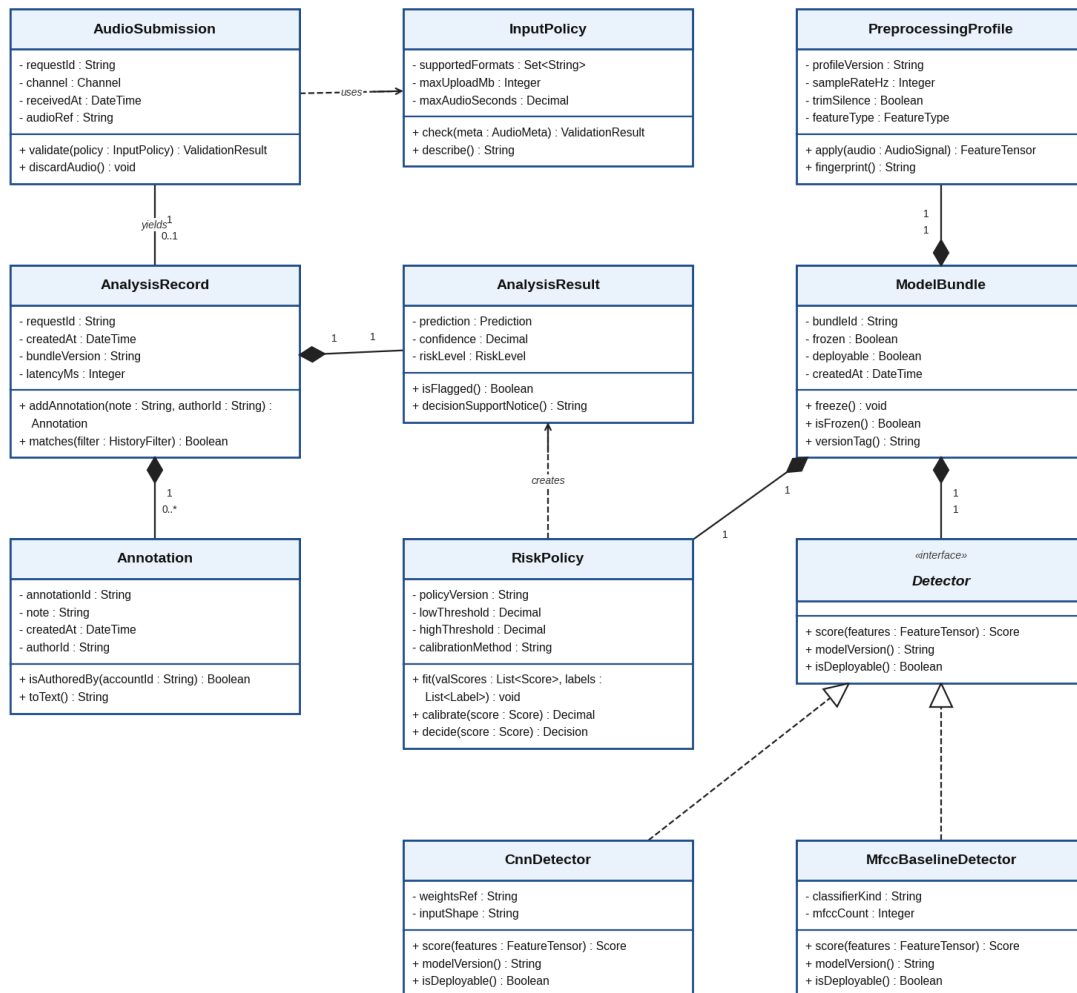
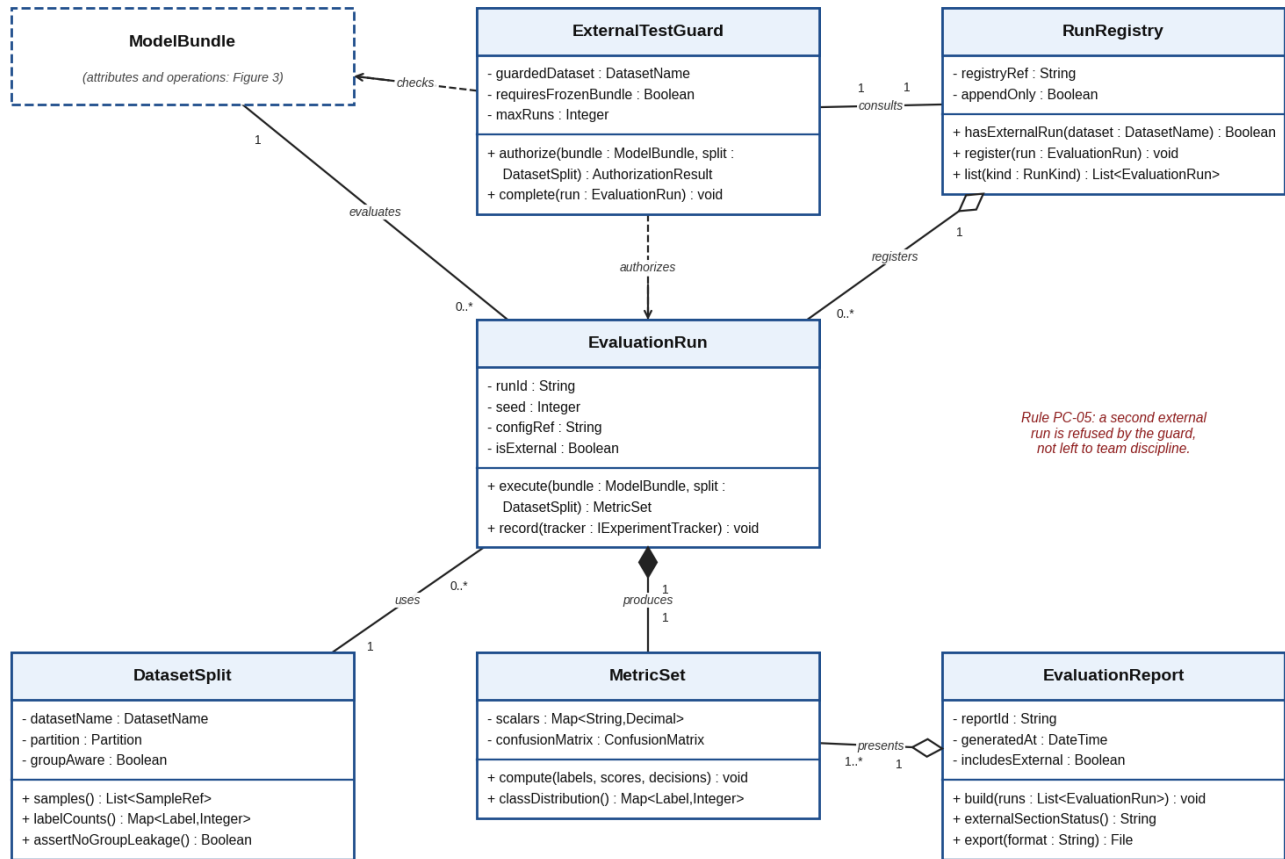


Figure 3 — Specification-level UML class diagram — analysis domain

A ModelBundle composes exactly one Detector, one PreprocessingProfile and one RiskPolicy, which makes NFR-09 (matching versions) a structural fact rather than a naming convention. Detector is an interface realized by CnnDetector and MfccBaselineDetector; *isDeployable()* is true only for the CNN (PC-08). AnalysisRecord composes an AnalysisResult and its Annotations, so an annotation can never modify the original result (FR-11), and it stores no audio (NFR-04).

Evaluation and research-governance domain — class diagram



Legend filled diamond = composition hollow diamond = aggregation dashed open arrow = dependency solid line = association

Figure 4 — UML class diagram — evaluation and research-governance domain

The ExternalTestGuard consults a RunRegistry and the ModelBundle's frozen flag before it authorizes an EvaluationRun on DEEP-VOICE; completing a run registers it, after which the guard refuses any further external run (PC-04, PC-05). MetricSet is composed by its EvaluationRun and aggregated by EvaluationReport, which can therefore state “external section not yet available” (FR-19).

3.2 Class Descriptions

Table 6 — Class design table (17 classes)

Class	Attributes	Operations	Related classes
AudioSubmission <i>One uploaded recording, from receipt through validation.</i>	requestId : String channel : Channel receivedAt : DateTime audioRef : String	validate(policy : InputPolicy) : ValidationResult discardAudio() : void	InputPolicy (dependency); AnalysisRecord (yields 0..1)
InputPolicy <i>Published limits every submission must satisfy, identical for dashboard and API.</i>	supportedFormats : Set<String> maxUploadMb : Integer maxAudioSeconds : Decimal	check(meta : AudioMeta) : ValidationResult describe() : String	AudioSubmission
PreprocessingProfile <i>Versioned recipe turning audio into model features, shared by training and inference.</i>	profileVersion : String sampleRateHz : Integer trimSilence : Boolean featureType : FeatureType	apply(audio : AudioSignal) : FeatureTensor fingerprint() : String	ModelBundle (composition)
Detector «interface» <i>Contract for any synthetic-speech scorer.</i>	—	score(features : FeatureTensor) : Score modelVersion() : String isDeployable() : Boolean	CnnDetector, MfccBaselineDetector (realizations); ModelBundle (composition)
CnnDetector <i>Primary deep model: a CNN over log-mel spectrograms.</i>	weightsRef : String inputShape : String	score(features : FeatureTensor) : Score modelVersion() : String isDeployable() : Boolean	Detector (realizes)
MfccBaselineDetector <i>Classical baseline over MFCC statistics, used for evaluation only.</i>	classifierKind : String mfccCount : Integer	score(features : FeatureTensor) : Score modelVersion() : String isDeployable() : Boolean	Detector (realizes)
RiskPolicy <i>Turns a raw score into prediction, confidence and risk level with versioned thresholds.</i>	policyVersion : String lowThreshold : Decimal highThreshold : Decimal calibrationMethod : String	fit(valScores : List<Score>, labels : List<Label>) : void calibrate(score : Score) : Decimal decide(score : Score) : Decision	ModelBundle (composition); AnalysisResult (creates)
ModelBundle <i>Binds detector, preprocessing profile and risk policy; immutable once frozen.</i>	bundleId : String frozen : Boolean deployable : Boolean createdAt : DateTime	freeze() : void isFrozen() : Boolean versionTag() : String	Detector, PreprocessingProfile, RiskPolicy (composition); EvaluationRun
AnalysisResult <i>The outcome returned to the caller and shown to the analyst.</i>	prediction : Prediction confidence : Decimal riskLevel : RiskLevel	isFlagged() : Boolean decisionSupportNotice() : String	AnalysisRecord (composition); RiskPolicy
AnalysisRecord <i>Persisted metadata of one successful analysis, never raw audio.</i>	requestId : String createdAt : DateTime bundleVersion : String latencyMs : Integer	addAnnotation(note : String, authorId : String) : Annotation matches(filter : HistoryFilter) : Boolean	AnalysisResult, Annotation (composition); AudioSubmission
Annotation <i>Analyst note stored beside, never inside, the original result.</i>	annotationId : String note : String createdAt : DateTime authorId : String	isAuthoredBy(accountId : String) : Boolean toText() : String	AnalysisRecord
DatasetSplit <i>Labeled partition of a dataset with its class counts and leakage status.</i>	datasetName : DatasetName partition : Partition groupAware : Boolean	samples() : List<SampleRef> labelCounts() : Map<Label,Integer> assertNoGroupLeakage() : Boolean	EvaluationRun
EvaluationRun <i>One reproducible execution of a bundle on a split.</i>	runId : String seed : Integer configRef : String isExternal : Boolean	execute(bundle : ModelBundle, split : DatasetSplit) : MetricSet record(tracker : IExperimentTracker) : void	ModelBundle, DatasetSplit, MetricSet (composition); RunRegistry
MetricSet <i>The required metrics for one run.</i>	scalars : Map<String,Decimal> confusionMatrix : ConfusionMatrix	compute(labels, scores, decisions) : void classDistribution() : Map<Label,Integer>	EvaluationRun (composition); EvaluationReport
ExternalTestGuard <i>Protection point that decides whether the DEEP-VOICE test may run.</i>	guardedDataset : DatasetName requiresFrozenBundle : Boolean maxRuns : Integer	authorize(bundle : ModelBundle, split : DatasetSplit) : AuthorizationResult complete(run : EvaluationRun) : void	RunRegistry; ModelBundle; EvaluationRun
RunRegistry <i>Append-only memory of experiment runs, used to enforce the exactly-once rule.</i>	registryRef : String appendOnly : Boolean	hasExternalRun(dataset : DatasetName) : Boolean register(run : EvaluationRun) : void list(kind : RunKind) : List<EvaluationRun>	ExternalTestGuard; EvaluationRun (aggregation)

Class	Attributes	Operations	Related classes
EvaluationReport <i>Assembled report with a labeled external-generalization section.</i>	reportId : String generatedAt : DateTime includesExternal : Boolean	build(runs : List<EvaluationRun>) : void externalSectionStatus() : String export(format : String) : File	MetricSet (aggregation)

3.3 Object-Oriented Quality

- **Single responsibility:** intake, preprocessing, detection, risk decision, recording, logging and persistence are separate components; RiskPolicy holds decision rules, not the detector; ExternalTestGuard holds the test-access rule, not EvaluationRun.
- **Encapsulation:** state changes go through meaningful operations (freeze, register, addAnnotation, discardAudio), never through public fields; a frozen ModelBundle rejects change.
- **Composition over reference:** ModelBundle owns its three parts and AnalysisRecord owns its result and annotations; shared concepts such as ModelBundle in EvaluationRun use ordinary associations.
- **Polymorphism:** callers depend on the Detector interface, so the same harness scores the baseline and the CNN (FR-12) and a parked model could be added without changing callers.
- **Dependency inversion:** services depend on repository, registry and tracker interfaces, so tests use in-memory doubles and CI needs no GPU, database server or external account (NFR-11).
- **Open/closed:** a new metric is a new key in MetricSet.scalars; a new dataset source is a new adapter behind the Dataset Manager.

4. Data Design

4.1 Data Model

Persistent analysis data is deliberately small: the system keeps **metadata about analyses, never the audio** (FR-09, NFR-04). Research artifacts (bundle manifests, run registry, dataset manifests) are append-only files versioned with the experiments. Column types are logical and implementation-neutral. PK = primary key, FK = foreign key.

Table 7 — Logical data model

Store / table	Fields	Keys and constraints
analysis_record	request_id, created_at, model_config_version, prediction, confidence, risk_level, latency_ms	PK request_id. prediction in {authentic, synthetic, uncertain}; confidence in [0,1]; risk_level in {LOW, MEDIUM, HIGH}. No audio and no personal-identifier columns (NFR-04).
annotation	annotation_id, request_id, author_id, note, created_at	PK annotation_id; FK request_id to analysis_record. Insert-only; never updates the analysis record (FR-11).
service_log entry	timestamp, request_id, model_config_version, latency_ms, outcome	outcome in {ok, validation_error, service_error}. No audio, no personal identifiers (FR-20, NFR-04).
temporary audio	audio_ref, received_at, expires_at	expires_at = received_at + RETENTION_HOURS; removed on release or by the retention job (NFR-03).
model_bundle manifest	bundle_id, model_version, profile_version, risk_policy_version, frozen, deployable, created_at, checkpoint_ref	PK bundle_id. A frozen manifest is immutable; deployable only if the detector's isDeployable() is true (PC-04, PC-08).
run_registry (append-only)	run_id, run_kind, dataset_name, bundle_id, seed, config_ref, executed_at, metrics_ref	PK run_id. At most one row with run_kind = external per dataset_name (PC-05).
dataset manifest	sample_id, path, label, dataset_name, partition, group_id (if available)	group_id present only where source metadata allows; otherwise the limitation is recorded (PC-02).

Table 8 — Data integrity rules

Data set	Integrity rule
Analysis record and annotation	An annotation is stored beside the analysis record and never changes its prediction, confidence or risk level.
Model bundle	Detector, preprocessing profile and risk policy carry one version tag; a mismatch blocks activation (NFR-09).
Frozen bundle	After freeze no weight, profile or threshold changes; any change creates a new bundle that must be frozen again before an external test.
External test	DEEP-VOICE results are produced once, from a frozen bundle, and are never used to retrain, retune or move a threshold (PC-05).
Dataset splits	FoR-2sec partitions are group-aware where metadata allows; DEEP-VOICE is never part of any training, validation or tuning partition (PC-02, PC-03).
Logs and records	No raw audio and no direct personal identifiers in analysis metadata or service logs.

4.2 Key Data Structures

Table 9 — Key data structures and their purpose

Structure	Purpose and design requirement	Requirements
History index (created_at descending; request_id)	Serves the most-recent-first history and request-ID lookup without scanning all records.	FR-10, NFR-07
Unique partial index on run_registry(dataset_name) where run_kind = external	Makes a second external run impossible at the storage level, not only in code.	PC-05
Bundle manifest with three version fields and a checkpoint reference	Binds model, preprocessing profile and risk policy so they load together.	NFR-09, PC-07
Dataset manifest with partition and optional group_id	Enables the leakage check and the group-aware split; records what metadata was missing.	PC-01, PC-02
Retention index on expires_at	Lets the retention job select expired raw audio without a full scan.	NFR-03
Confusion matrix with rows {bonafide, spoof} and columns {authentic, synthetic, uncertain}	Shows false negatives, false positives and correct deferrals explicitly.	FR-13

4.3 Data Security and Lifecycle

Uploaded audio is handled as sensitive: it is held in the Temporary Audio Store only for the duration of the analysis or until RETENTION_HOURS, and is never copied to logs or records. Annotation author IDs identify analyst accounts, not callers. The datasets (including recordings of real public figures in DEEP-VOICE) stay on the shared Drive; the repository holds manifests and code only, and dataset licenses are checked in the Week 2 audit (PC-01). Retention length, deletion mechanism and history persistence are configuration values, not design constants (Section 9.5).

5. Interface Design

5.1 User Interface Overview

The dashboard is organized by user goal, not by internal component. Every result screen shows prediction, confidence, risk level and the decision-support notice together (FR-08), and never presents an *uncertain* result as a fraud finding (NFR-05).

Table 10 — Interface areas by role

Role	Interface areas	Requirements
Fraud Prevention Analyst	Upload and result page; analysis history with flagged markers; annotation	FR-01, FR-08, FR-10, FR-11, NFR-05
API Client	REST endpoint POST /analyze-audio with the same limits and logic as the dashboard	FR-02, NFR-06, NFR-08
Compliance Officer	Evaluation report page (in-domain results and a labeled external section)	FR-13 to FR-16, FR-19
Operations / Security Administrator	Service-log and monitoring view	FR-20, NFR-02, NFR-04
ML / Research Engineer	Research notebooks (no web interface): audit, training, evaluation, external test	UC-07, UC-08, PC-01 to PC-08

5.2 API and Interface Contracts

The table lists the provided interface of every component with representative operations. Sequence messages in Section 6 use exactly these operation names.

Table 11 — Interface contracts

Interface (component)	Representative operations	Result and error contract
IUserExperience Analyst Dashboard	selectAudioFile, viewHistory, addAnnotation, viewEvaluationReport	Renders prediction, confidence, risk level and decision-support notice together; shows validation and service errors as readable messages; never presents an 'uncertain' result as a fraud finding.
IAnalysisAPI API Gateway	POST /analyze-audio, GET /history, POST /annotations, GET /reports/evaluation, GET /admin/service-logs	JSON with a request ID. Validation and authorization failures return structured 4xx errors, service failures a structured 5xx error; no stack traces, weights, secrets or training-data locations.
IAccessControl Access Control	authenticate, authorize	Returns account and role, or a denial. Annotation and service-log actions are refused without an authenticated account of the right role.
IAnalysisService Analysis Service	analyze	Returns an AnalysisResult or a validation/service error. On any error no record is written and no prediction is fabricated.
IAudioIntake Audio Intake Service	accept, release, purgeExpired	Returns an AudioSubmission or a ValidationError naming the violated policy; raw audio is removed on release or after RETENTION_HOURS.
IPreprocessing Preprocessing Service	transform	Returns the FeatureTensor for the requested profile version; unreadable audio raises a validation error.
IDetection Detector Runtime	activateBundle, score	Returns a raw Score from the loaded model; refuses to activate a bundle whose model, profile and risk-policy versions do not match.
IRiskScoring Risk & Confidence Service	decide	Returns exactly one Decision: prediction in {authentic, synthetic, uncertain}, confidence in [0,1], risk level in {LOW, MEDIUM, HIGH}.
IAnalysisRecord Analysis Record Service	record, history, annotate	Stores seven metadata fields without raw audio; history is most-recent-first with flagged markers; an annotation is stored beside the unchanged result.
IOperationalLog Operational Log	logEvent, view	Returns version, volume, latency and error-rate entries; entries never contain raw audio or personal identifiers.
IAnalysisRepository Repository Layer	save, query, saveAnnotation	Transactional. An unavailable store yields a clear retrieval error, never stale or fabricated data.
IResearchWorkflow Research Notebooks	runDatasetAudit, runTraining, runEvaluation, runExternalTest	Notebook-level entry points; every call returns a run ID and a result summary.
ITraining Training & Tuning Pipeline	trainBaseline, trainCnn, tuneRiskPolicy, assembleBundle	Uses FoR-2sec partitions only; returns a candidate ModelBundle; refuses to start before the dataset audit is documented (PC-01).
IEvaluation Evaluation Service	runEvaluation, runExternalTest, getReport	Returns a MetricSet and EvaluationRun; getReport marks the external section 'not yet available' until an external run exists.
IExternalTest External Test Guard	authorize	Returns authorized or refused(reason); refuses when the bundle is not frozen or an external run is already registered.
IDatasetManager Dataset Manager	auditDataset, buildGroupAwareSplit, loadSplit	Returns manifests, class counts and leakage status; records a limitation when group metadata is unavailable.
IModelRegistry Model Registry	saveBundle, loadBundle, freeze, isFrozen	Bundles are versioned; a frozen bundle rejects any further change.
IExperimentTracker Experiment Tracker	logRun, registerExternalRun, externalRunExists	Every run stores config, seed, dataset reference and metrics; registerExternalRun is append-only.

Illustrative API exchange (values are examples of the contract, not measured results):

```
POST /analyze-audio (multipart/form-data: file)
200 OK
{ "request_id": "7c1e...", "prediction": "uncertain", "confidence": 0.62, "risk_level": "MEDIUM",
  "model_config_version": "cnn-v1 / profile-v1 / risk-v1",
  "notice": "Decision support only - not proof of fraud." }
4xx { "error": "validation_error", "reason": "unsupported format" } (no inference, no record)
5xx { "error": "service_error", "request_id": "7c1e..." } (no stack trace, no fabricated prediction)
```

6. Behavioral Design

6.1 Sequence Diagrams for Key Use Cases

We modeled the two **Must-have** use cases that carry the project's core promise: the runtime analysis flow and the research-governance flow. Both use the components and operation names of Sections 2 and 5. Figures 5 and 6 follow on landscape pages; the numbered message lists in the Lab 5 worksheet format are in Section 9.6.

6.1.1 UC-01 Analyze Audio via Dashboard (includes UC-03 Validate & Analyze Audio and UC-04 Record Analysis Metadata)

Figure 5 realizes UC-01 for the Fraud Prevention Analyst. The dashboard forwards the file to the API Gateway; the Analysis Service validates it through the Audio Intake Service and, only if it is valid, runs preprocessing, detection and the risk decision, records metadata without audio, releases the raw audio and returns the result. The alternative path returns a structured validation error without inference and without a record. The traceable Must-have requirements are FR-01, FR-03 to FR-06, FR-08 and FR-09.

6.1.2 UC-08 Run External Generalization Test

Figure 6 realizes UC-08 for the ML / Research Engineer, who triggers it from a research notebook. Before any DEEP-VOICE data is read, the External Test Guard checks that the bundle is frozen (Model Registry) and that no external run exists (Experiment Tracker). Only then does the Evaluation Service load the frozen bundle, score the external split with the same preprocessing and frozen thresholds, compute the metric set and register the run, which closes the door on any second external run. The Must-have requirements are FR-13 and FR-15, with PC-04 and PC-05.

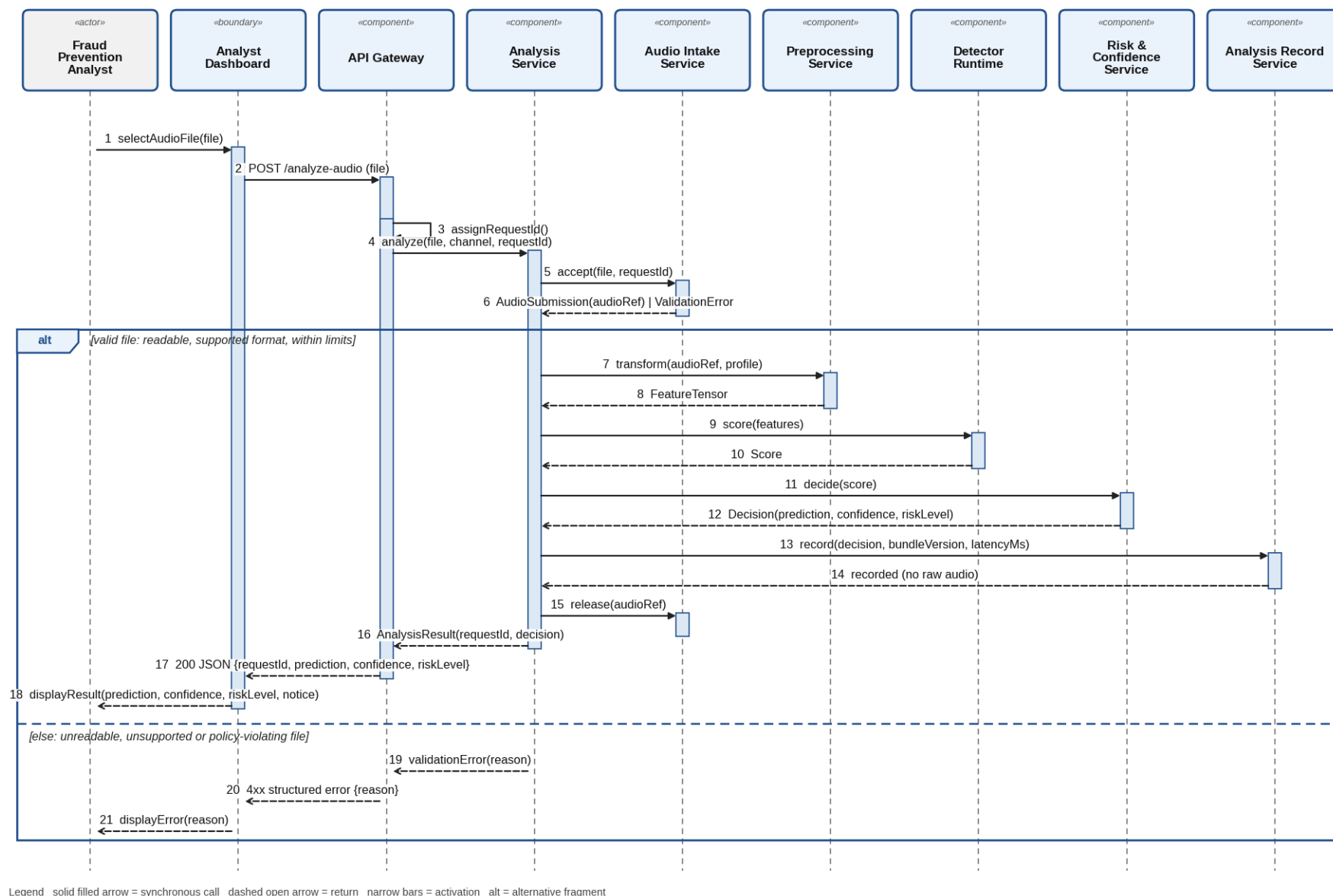


Figure 5 — UML sequence diagram for UC-01 Analyze Audio via Dashboard

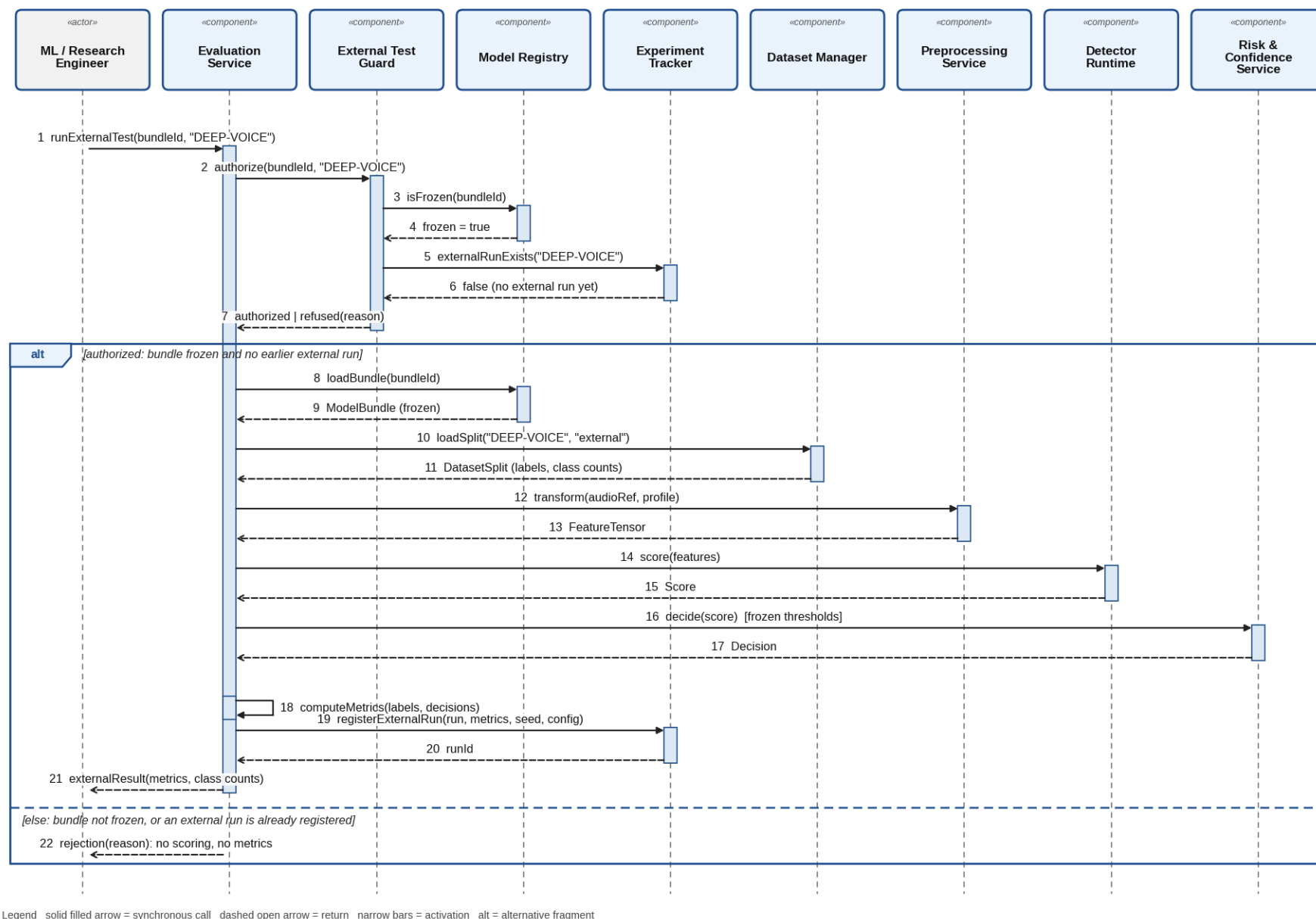


Figure 6 — UML sequence diagram for UC-08 Run External Generalization Test

6.1.3 Reading the sequences

- **Figure 5** shows why the dashboard has no detector logic: messages 2–4 send every submission through the API Gateway (NFR-08), and the Analysis Service is the only participant that orders the pipeline. Messages 13–15 record metadata (no raw audio, FR-09, NFR-04) and release the audio (NFR-03). In the *else* branch nothing after message 6 executes, which is how NFR-06 (no fabricated success) is met.
- **Figure 6** shows the guard pattern: messages 2–7 happen before the DEEP-VOICE split is loaded, and message 19 registers the run so the guard's check at message 5 fails for any later attempt. Messages 12–17 reuse the shared Preprocessing Service, Detector Runtime and Risk & Confidence Service, so the external result is measured on exactly the pipeline that would be deployed.
- Both diagrams use only participants from Figures 1–2 and only operations from Table 11 (checked in Section 9.2).

6.2 Additional Behavioral Rules

The remaining use cases follow the same interfaces and are specified as behavioral rules rather than separate diagrams.

Table 12 — Behavioral rules beyond the two principal sequence diagrams

Behavior	Trigger	Design response
UC-02 Analyze Audio via API	API Client sends POST /analyze-audio	Same path as Figure 5 from message 4 onward; returns JSON or a structured 4xx/5xx error; no stack trace (NFR-02, NFR-06).
UC-05 Review Analysis History	Analyst opens history	Analysis Record Service returns records most recent first; results labeled synthetic or uncertain carry a flagged marker (AnalysisResult.isFlagged); empty state if none (FR-10).
UC-06 Annotate Result	Analyst adds a note	Access Control verifies an authenticated analyst account; the service stores an Annotation beside the unchanged record and logs the event; otherwise the request is denied (FR-11).
UC-07 Run Model Evaluation	Engineer starts evaluation from a notebook	Evaluation Service runs the MFCC baseline and the CNN through one harness on the same FoR-2sec test split, after the audit and split checks (PC-01, PC-02), and computes the full MetricSet (FR-12, FR-13, FR-14).
UC-09 View Evaluation Report	Compliance Officer requests the report	EvaluationReport assembles in-domain results and adds the external section only if a registered external run exists; otherwise it says “not yet available” (FR-16, FR-19).
UC-10 Review Service Logs	Administrator opens the log view	Operational Log returns version, volume, latency and error rate with no audio or personal data; a connectivity error is shown instead of stale data (FR-20).
Raw-audio retention	Scheduled job, at least hourly	Audio Intake Service purges audio older than RETENTION_HOURS; the analysis record stays intact (NFR-03).
Service start-up	Container starts	Detector Runtime loads the configured bundle via the Model Registry and refuses to start if the model, profile and risk-policy versions do not match (NFR-09).
Training and tuning	Engineer starts training	Pipeline refuses to start before the dataset audit is documented; fits models and the risk policy on FoR-2sec partitions only, assembles a bundle and freezes it when selection is final (PC-01, PC-03, PC-04).
Stretch goals	Engineer runs degradation or calibration	Evaluation Service applies controlled degradation through the Preprocessing Service (FR-17) and fits calibration on the validation split, reporting a reliability diagram (FR-18). No other extension is designed (PC-09).

7. Design Rationale

7.1 Design Patterns Used and Why

Table 13 — Design pattern justification

Pattern	Problem solved	Where applied	Justification
Strategy	Two interchangeable detectors (MFCC baseline, CNN) must run through one harness and only one may be deployed; a parked model must be addable later.	Detector interface with CnnDetector and MfccBaselineDetector; used by Detector Runtime, Training & Tuning Pipeline and Evaluation Service.	Callers depend on score(), not on model type (FR-12, PC-08). isDeployable() encodes the deployment rule.
Facade	Dashboard and API both need “validate and analyze”, which involves five collaborators in a fixed order; duplicating the orchestration would let channels diverge.	Analysis Service exposes analyze(); the API Gateway calls nothing else in the pipeline.	One definition of the pipeline order for every channel (UC-03, NFR-08, NFR-06).
Adapter	Provider-specific interfaces (Weights & Biases, Kaggle / Hugging Face, Google Drive) would leak into research code.	Experiment Tracker, Dataset Manager and Model Registry storage adapters.	Providers can change or be faked in tests without touching core code (PC-07, NFR-11).
Repository	Services need persistence without SQL details, and tests need an in-memory replacement.	Repository Layer behind IAnalysisRepository, used by the Analysis Record Service.	Isolated tests and a replaceable store (FR-09, NFR-11).
Factory	Online and research tiers must build a matching detector, preprocessing profile and risk policy from one bundle ID; building the parts separately allows mismatched versions.	Model Registry.loadBundle() returns a ModelBundle that composes the three parts.	Version consistency by construction (NFR-09).
Protection Proxy	The DEEP-VOICE test must run only on a frozen model and only once; relying on team discipline is fragile.	External Test Guard in front of the Evaluation Service's external evaluation, backed by RunRegistry.	Access to the external dataset is controlled in code (FR-15, PC-04, PC-05).

7.2 Alternatives Considered and Trade-offs

The full methodology comparison is Table 3. Four further alternatives deserve explicit trade-off statements.

- **Microservices for detector and evaluation.** Independent deployment would add service discovery, distributed tracing and deployment automation for a workload of short synchronous requests. The modular layered design keeps clear boundaries, so a component can be extracted later if measurements justify it.
- **Embedding the detector in the dashboard.** Faster to build, but it creates a second code path: the dashboard and the API could disagree on prediction or risk level and processing would be coupled to the UI. Routing the dashboard through the API costs one HTTP hop.
- **Enforcing the one-time external test by documentation only.** Zero code, but one accidental re-run or re-tune silently invalidates the project's central research claim. The guard and registry cost two small classes.
- **Runtime ensemble of baseline and CNN.** Might raise scores, but it contradicts PC-08 and multiplies the evaluation surface; the baseline stays an evaluation reference.

8. Requirements Traceability Matrix

Table 14 maps every one of the 40 SRS requirements to the components and classes that realize it and to the evidence in this document. Requirement wording is abbreviated from SRS Section 7.2.

Table 14 — Requirements to design traceability

ID	Requirement	Design elements (components; classes)	Evidence
FR-01	Analyst uploads audio on the dashboard	Analyst Dashboard; API Gateway Classes: <i>AudioSubmission</i>	Fig. 5
FR-02	API Client submits via POST /analyze-audio	API Gateway Classes: <i>AudioSubmission</i>	Table 11, Fig. 5
FR-03	Reject invalid files with a reason	Audio Intake Service Classes: <i>InputPolicy</i> , <i>AudioSubmission</i>	Fig. 5 (alt branch)
FR-04	Versioned preprocessing profile	Preprocessing Service Classes: <i>PreprocessingProfile</i> , <i>ModelBundle</i>	Fig. 3, Fig. 5
FR-05	Exactly one three-way prediction	Risk & Confidence Service; Detector Runtime Classes: <i>RiskPolicy</i> , <i>AnalysisResult</i>	Fig. 5
FR-06	Numeric confidence in [0,1]	Risk & Confidence Service Classes: <i>RiskPolicy</i> , <i>AnalysisResult</i>	Table 11
FR-07	LOW/MEDIUM/HIGH risk level (versioned policy)	Risk & Confidence Service Classes: <i>RiskPolicy</i> , <i>ModelBundle</i>	Fig. 3
FR-08	Show prediction, confidence, risk level and notice	Analyst Dashboard Classes: <i>AnalysisResult</i>	Fig. 5
FR-09	Log every analysis, no raw audio	Analysis Record Service; Repository Layer Classes: <i>AnalysisRecord</i>	Fig. 5, Table 7
FR-10	History view with flagged marker	Analysis Record Service; Analyst Dashboard Classes: <i>AnalysisRecord</i> , <i>AnalysisResult</i>	Table 12
FR-11	Access-controlled analyst annotation	Access Control; Analysis Record Service Classes: <i>Annotation</i> , <i>AnalysisRecord</i>	Table 12
FR-12	Baseline vs CNN on one harness	Evaluation Service; Detector Runtime Classes: <i>Detector</i> , <i>CnnDetector</i> , <i>MfccBaselineDetector</i> , <i>EvaluationRun</i>	Fig. 3, Table 12
FR-13	Full metric set incl. confusion matrix	Evaluation Service Classes: <i>MetricSet</i>	Fig. 4, Fig. 6
FR-14	Dataset/split and class distribution stated	Evaluation Service; Dataset Manager Classes: <i>DatasetSplit</i> , <i>EvaluationReport</i>	Fig. 4
FR-15	Frozen model on DEEP-VOICE, exactly once	External Test Guard; Evaluation Service Classes: <i>ExternalTestGuard</i> , <i>RunRegistry</i>	Fig. 6
FR-16	Gap and caveats reported with the result	Evaluation Service Classes: <i>EvaluationReport</i> , <i>MetricSet</i>	Fig. 6, Table 12
FR-17	Degradation curve (stretch goal)	Evaluation Service; Preprocessing Service Classes: <i>EvaluationRun</i>	Table 12
FR-18	Reliability diagram / ECE (stretch goal)	Evaluation Service; Risk & Confidence Service Classes: <i>RiskPolicy</i> , <i>MetricSet</i>	Table 12
FR-19	Placeholder if external test not yet run	Evaluation Service Classes: <i>EvaluationReport</i>	Table 12
FR-20	Service-log review for the administrator	Operational Log; API Gateway	Table 12
NFR-01	95% of requests within L_MAX	Analysis Service; Detector Runtime; Operational Log	Section 2.4, Table 5
NFR-02	No secrets, weights or stack traces exposed	API Gateway; Access Control	Table 11
NFR-03	Raw-audio retention up to RETENTION_HOURS	Audio Intake Service Classes: <i>AudioSubmission</i>	Fig. 5 (release), Table 12
NFR-04	No raw audio or personal identifiers in logs	Analysis Record Service; Operational Log Classes: <i>AnalysisRecord</i>	Table 7
NFR-05	Decision-support framing everywhere	Analyst Dashboard Classes: <i>AnalysisResult</i>	Fig. 5, Table 10
NFR-06	Structured error, no fabricated success	API Gateway; Analysis Service	Fig. 5 (alt branch)
NFR-07	Traceable by request ID, version, latency	API Gateway; Analysis Record Service Classes: <i>AnalysisRecord</i> , <i>ModelBundle</i>	Table 7

ID	Requirement	Design elements (components; classes)	Evidence
NFR-08	Dashboard and API share one input policy	Analyst Dashboard; API Gateway; Audio Intake Service <i>Classes: InputPolicy</i>	Fig. 1, Fig. 5
NFR-09	Model, profile and risk config versioned together	Model Registry; Detector Runtime <i>Classes: ModelBundle</i>	Fig. 3
NFR-10	Containers deployable from the README	API Gateway; Analyst Dashboard	Section 2.4, Table 5
NFR-11	Automated tests in CI	Repository Layer	Section 3.3, Table 13
PC-01	Dataset audit before training	Dataset Manager; Training & Tuning Pipeline <i>Classes: DatasetSplit</i>	Table 11
PC-02	Group-aware split or documented limitation	Dataset Manager <i>Classes: DatasetSplit</i>	Fig. 4
PC-03	Tune on FoR-2sec only	Training & Tuning Pipeline; Risk & Confidence Service <i>Classes: RiskPolicy</i>	Fig. 3
PC-04	Freeze before the external test	Model Registry; External Test Guard <i>Classes: ModelBundle</i>	Fig. 6
PC-05	DEEP-VOICE evaluated exactly once	External Test Guard; Experiment Tracker <i>Classes: RunRegistry, ExternalTestGuard</i>	Fig. 4, Fig. 6
PC-06	Required metric set, no t-DCF	Evaluation Service <i>Classes: MetricSet</i>	Fig. 4
PC-07	Reproducible runs	Experiment Tracker; Model Registry <i>Classes: EvaluationRun</i>	Table 7
PC-08	CNN is the single required model	Detector Runtime; Training & Tuning Pipeline <i>Classes: Detector, ModelBundle</i>	Fig. 3
PC-09	Exactly two stretch goals	Scope control (no design element)	Scope control; hooks in FR-17/FR-18 rows

9. Appendices

9.1 Design Quality Checklist

Table 15 — Design quality checklist

Status	Validation item	Evidence
PASS	Every SRS requirement is addressed by at least one design element.	Table 14: all 40 identifiers (FR-01–FR-20, NFR-01–NFR-11, PC-01–PC-09) map to named components or classes.
PASS	Each component has one principal responsibility and a named provided interface.	Table 4 (18 components), Figures 1–2.
PASS	Component dependencies contain no cycles and point toward provided interfaces.	“Talks to” column of Table 4 checked as a directed graph; Figures 1–2.
PASS	Every class has 2–3 key operations; the model uses multiplicities, composition, aggregation and realization; no foreign-key attributes.	Table 6, Figures 3–4; keys appear only in Table 7.
PASS	Sequence participants exist in the component model and every call uses an operation of the receiver’s provided interface.	Figures 5–6 against Table 4 and Table 11 (Section 9.2).
PASS	Research-integrity constraints are enforced structurally, not by convention.	Frozen ModelBundle, External Test Guard, RunRegistry (Figures 4 and 6, Table 7).
PASS	Security, privacy, reliability and traceability requirements are reflected in the design.	NFR-02 to NFR-07 rows of Table 14; Sections 4.3 and 5.2.
PASS	Where the SRS leaves choices open, the design stays replaceable behind interfaces.	Access Control, Repository Layer, Adapters; open values listed in Section 9.5.
PASS	Every diagram has a title, numbered caption, an introduction before it and an explanation after it.	Figures 1–6 and Sections 2.2, 3.1, 6.1.
OPEN	Peer-review exchange with another team, revisions from it and instructor sign-offs.	To be completed in class: Tables 18 and 22.
OPEN	Operational parameters not yet baselined (L_MAX, upload and duration limits, retention, risk thresholds).	Named parameters tracked in Section 9.5; latency benchmark in Week 2.
OPEN	No implementation evidence yet: this is a design-level validation.	Test cases are assigned in the Testing phase (Week 13), as in the SRS.

9.2 Cross-Diagram Consistency

Table 16 — Cross-diagram consistency review

Consistency check	Result
Names	Component, class and interface names are identical in the figures, Tables 4, 6 and 11 and the sequence lists (tables and figures are generated from one design model).
Actors	UC-01 uses the Fraud Prevention Analyst; UC-08 uses the ML / Research Engineer — both defined in the SRS.
Participants	All 17 sequence participants (Figures 5–6) are actors or components of Figures 1–2.
Operations	Every call message in Figures 5–6 is an operation of the receiver’s provided interface (Table 11).
Dependencies	Service calls follow the component direction of Figures 1–2; the core never depends on dashboard or notebook code.
Requirement coverage	All 40 requirements appear in Table 14; every requirement reference inside Table 4 exists in the SRS.

9.3 Internal Pre-Review Log

Before the in-class exchange we reviewed our own draft against the Lab 4 design quality checklist. These are the design flaws found and corrected; they are **not** peer feedback (see 9.4).

Table 17 — Internal pre-review log

#	Issue found	Why it matters	Correction applied	Status
1	First sketch let the dashboard call the detector directly.	Two code paths could disagree with the API on prediction or risk level (NFR-08, UN-05).	Dashboard now calls only the API Gateway; Analysis Service is the single facade.	Fixed
2	Preprocessing was drawn separately for training and for serving.	Training/serving skew would make offline metrics describe a different system (FR-04, NFR-09).	One Preprocessing Service and PreprocessingProfile, versioned inside the ModelBundle, shared by both tiers.	Fixed
3	Nothing in the architecture enforced the frozen, once-only DEEP-VOICE test.	PC-04 and PC-05 would depend on memory and discipline.	Added ExternalTestGuard, RunRegistry and the frozen flag; UC-08 sequence shows the checks before any data is read.	Fixed
4	Risk thresholds lived inside the detector.	Mixes model and policy; blocks separate versioning and FoR-only tuning (FR-07, PC-03).	Extracted RiskPolicy as its own class and component, composed in the bundle.	Fixed
5	One logging component served analysis metadata and service logs.	FR-20 needs a separate operational view; NFR-04 forbids audio and personal data in both.	Split Analysis Record Service (metadata, history, annotation) from Operational Log.	Fixed
6	The MFCC baseline appeared deployable.	PC-08 makes the CNN the only deployed model.	Detector.isDeployable() and ModelBundle.deployable; baseline exists for evaluation only.	Fixed
7	Raw uploads had no owner for retention.	NFR-03 needs deletion after RETENTION_HOURS.	Audio Intake Service owns the Temporary Audio Store and its purge job; release() after analysis.	Fixed

9.4 Peer Feedback Received (in-class exchange)

Complete during Step 6 of the practical session: swap the architecture, class and sequence diagrams with another team, record at least three issues they find in our design using the Lab 4 checklist, and describe how we revised the design. Rows are intentionally blank in this pre-session version.

Table 18 — Peer feedback received and revisions made

Reviewing team	Issue found	Our fix

9.5 Assumptions and Open Decisions

- **Parameters** stay named, as in SRS 7.4, and have one design home each: L_MAX (Analysis Service and Operational Log, baselined by the Week 2 benchmark); MAX_UPLOAD_MB, MAX_AUDIO_SECONDS and SUPPORTED_AUDIO_FORMATS (InputPolicy, shared by dashboard and API); RETENTION_HOURS (Audio Intake retention job); risk thresholds (RiskPolicy, fitted on the FoR-2sec validation split only and frozen with the bundle).
- **Clips longer than 2 seconds** (SRS 7.4 flags that the model is trained on 2-second clips): reject, truncate or window-and-aggregate is undecided; it is a Preprocessing Service and InputPolicy decision to be taken in Phase 2 together with the latency benchmark.
- **Access-control mechanism** (API key or analyst login) is open; Access Control hides it behind IAccessControl. Whether analysis submission itself is authenticated is part of the same decision.
- **History persistence** across restarts and sharing between analysts is open; the Repository Layer starts with a SQLite volume and can change without touching services.

- **Dataset facts** (class counts, sizes, speakers, TTS engines actually present in the 2-second packaging) are not asserted here; they are established by the Week 2 audit from the dataset files themselves (PC-01).
- **Hosting vendor** and the exact TLS termination remain deployment decisions (SRS 1.2).

9.6 Sequence Templates (Lab 5 worksheet format)

Sequence for Use Case: UC-01 Analyze Audio via Dashboard

Actors/Objects involved: Fraud Prevention Analyst (actor); Analyst Dashboard (boundary); API Gateway, Analysis Service, Audio Intake Service, Preprocessing Service, Detector Runtime, Risk & Confidence Service, Analysis Record Service (components)

Table 19 — Message list for UC-01 (numbers match Figure 5)

#	Sender -> Receiver : Message
1	Fraud Prevention Analyst -> Analyst Dashboard : selectAudioFile(file)
2	Analyst Dashboard -> API Gateway : POST /analyze-audio (file)
3	API Gateway -> API Gateway : assignRequestId() (self)
4	API Gateway -> Analysis Service : analyze(file, channel, requestId)
5	Analysis Service -> Audio Intake Service : accept(file, requestId)
6	Audio Intake Service -> Analysis Service : AudioSubmission(audioRef) ValidationError (return) <i>alt [valid file: readable, supported format, within limits]</i>
7	Analysis Service -> Preprocessing Service : transform(audioRef, profile)
8	Preprocessing Service -> Analysis Service : FeatureTensor (return)
9	Analysis Service -> Detector Runtime : score(features)
10	Detector Runtime -> Analysis Service : Score (return)
11	Analysis Service -> Risk & Confidence Service : decide(score)
12	Risk & Confidence Service -> Analysis Service : Decision(prediction, confidence, riskLevel) (return)
13	Analysis Service -> Analysis Record Service : record(decision, bundleVersion, latencyMs)
14	Analysis Record Service -> Analysis Service : recorded (no raw audio) (return)
15	Analysis Service -> Audio Intake Service : release(audioRef)
16	Analysis Service -> API Gateway : AnalysisResult(requestId, decision) (return)
17	API Gateway -> Analyst Dashboard : 200 JSON {requestId, prediction, confidence, riskLevel} (return)
18	Analyst Dashboard -> Fraud Prevention Analyst : displayResult(prediction, confidence, riskLevel, notice) (return) <i>else [else: unreadable, unsupported or policy-violating file]</i>
19	Analysis Service -> API Gateway : validationError(reason) (return)
20	API Gateway -> Analyst Dashboard : 4xx structured error {reason} (return)
21	Analyst Dashboard -> Fraud Prevention Analyst : displayError(reason) (return)

Sequence for Use Case: UC-08 Run External Generalization Test

Actors/Objects involved: ML / Research Engineer (actor, working from a Research Notebook); Evaluation Service, External Test Guard, Model Registry, Experiment Tracker, Dataset Manager, Preprocessing Service, Detector Runtime, Risk & Confidence Service (components)

Table 20 — Message list for UC-08 (numbers match Figure 6)

#	Sender -> Receiver : Message
1	ML / Research Engineer -> Evaluation Service : runExternalTest(bundleId, "DEEP-VOICE")
2	Evaluation Service -> External Test Guard : authorize(bundleId, "DEEP-VOICE")
3	External Test Guard -> Model Registry : isFrozen(bundleId)
4	Model Registry -> External Test Guard : frozen = true (return)
5	External Test Guard -> Experiment Tracker : externalRunExists("DEEP-VOICE")
6	Experiment Tracker -> External Test Guard : false (no external run yet) (return)
7	External Test Guard -> Evaluation Service : authorized refused(reason) (return)
	<i>alt [authorized: bundle frozen and no earlier external run]</i>
8	Evaluation Service -> Model Registry : loadBundle(bundleId)
9	Model Registry -> Evaluation Service : ModelBundle (frozen) (return)
10	Evaluation Service -> Dataset Manager : loadSplit("DEEP-VOICE", "external")
11	Dataset Manager -> Evaluation Service : DatasetSplit (labels, class counts) (return)
12	Evaluation Service -> Preprocessing Service : transform(audioRef, profile)
13	Preprocessing Service -> Evaluation Service : FeatureTensor (return)
14	Evaluation Service -> Detector Runtime : score(features)
15	Detector Runtime -> Evaluation Service : Score (return)
16	Evaluation Service -> Risk & Confidence Service : decide(score) [frozen thresholds]
17	Risk & Confidence Service -> Evaluation Service : Decision (return)
18	Evaluation Service -> Evaluation Service : computeMetrics(labels, decisions) (self)
19	Evaluation Service -> Experiment Tracker : registerExternalRun(run, metrics, seed, config)
20	Experiment Tracker -> Evaluation Service : runId (return)
21	Evaluation Service -> ML / Research Engineer : externalResult(metrics, class counts) (return)
	<i>else [else: bundle not frozen, or an external run is already registered]</i>
22	Evaluation Service -> ML / Research Engineer : rejection(reason): no scoring, no metrics (return)

9.7 Lab 5 Submission Checklist and Instructor Checkpoints

Table 21 — Lab 5 end-of-session deliverables and where they appear

Deliverable (Lab 5 step)	Location in this document	Status
Methodology and architecture decision with justification (Step 1)	Section 2.1, Table 3 and decision box	Complete
Component/architecture diagram and responsibility table (Step 2)	Sections 2.2–2.3, Figures 1–2, Table 4	Complete
Class diagram and class design table (Step 3)	Sections 3.1–3.2, Figures 3–4, Table 6	Complete
Two sequence diagrams for Must-have use cases (Step 4)	Section 6.1, Figures 5–6; worksheet lists in 9.6	Complete
Design-pattern justification table (Step 5)	Section 7.1, Table 13	Complete
Peer review feedback log and revisions made (Step 6)	Section 9.4, Table 18 (internal pre-review in 9.3)	To complete in class
Draft SDD skeleton, Sections 1–2 completed (Step 6)	Sections 1–2 (the document continues through Section 9)	Complete
Instructor sign-off at each checkpoint	Table 22 below	To complete in class

Table 22 — Checkpoints and instructor sign-off

Checkpoint	Evidence in this document	Instructor sign-off / date
Step 1 — explain the methodology and style in one minute	Section 2.1	
Step 2 — show component diagram and table	Figures 1–2, Table 4	
Step 3 — instructor reviews at least 2 classes for cohesion and single responsibility	Table 6 (e.g. RiskPolicy, ExternalTestGuard)	
Step 4 — show the 2 sequence diagrams	Figures 5–6	
End of session — architecture, class and at least one sequence diagram	Figures 1–6	